$ ./lighthouse --event “nush hack”

nush.hack

lighthouse: offline-first cybersecurity checklists

Zhuoxuan, Yiming, Xiaofan

$ our team

Zhuoxuan

Coder

Xiaofan

Slides Designer

Yiming

Debugging

$ problem.txt

> Build an app that lets safety personnel keep completing security processes offline and syncs their changes correctly when internet access returns.

The hard parts: no lost work, no duplicates or conflicts on reconnect.

$ ./lighthouse --solve

1. work offline

Every step is saved to a local SQLite database and queued in an outbox. The app never needs the server to record progress.

2. sync correctly

Unique event ids make retries safe, a since token pulls only new events, and the outbox clears only after the server confirms.

3. trust the record

Completed wins when devices disagree, and a hash-chained audit log shows if anyone tampers with history.

$ ls lighthouse/

processes.py

Shared checklists: the data both sides use.

server.py

FastAPI + SQLite sync server and 4-page web dashboard.

app.py

PyQt6 desktop client with a local database.

run.py

One command to start the server and clients.

app.py <-- sync --> server.py, both importing processes.py

$ processes.py

- PROCESSES: 4 built-in checklists (ransomware response, phishing triage, patching, offboarding) with 22 steps in total.
- STEP_NAMES: flat lookup from step id to readable name.
- Imported by both app.py and server.py, so client and server never disagree.
- Step ids are unique, which is what makes syncing safe.
- Checklists added on the dashboard are not in this file. They live in server.db.

processes.py > PROCESSES

```python
PROCESSES = {
    "ir": {
        "name": "Incident Response (Ransomware)",
        "steps": [
            ("ir1", "Confirm the incident and open a case ticket"),
            ("ir2", "Isolate affected hosts from the network"),
            ("ir3", "Preserve logs and take forensic disk images"),
            ("ir4", "Reset compromised credentials and revoke sessions"),
            ("ir5", "Remove malware and close the attacker's access"),
            ("ir6", "Restore from clean backups and verify integrity"),
            ("ir7", "Write the post-incident report"),
        ],
    },
    "phish": {
        "name": "Phishing Report Triage",
        "steps": [
            ("ph1", "Pull the reported email with full headers"),
            ("ph2", "Detonate links and attachments in a sandbox"),
            ("ph3", "Block sender domain and malicious URLs"),
            ("ph4", "Search mailboxes and purge copies of the email"),
            ("ph5", "Reset passwords for anyone who clicked"),
        ],
    },
    "patch": {
        "name": "Monthly Patch & Vulnerability Check",
        "steps": [
            ("pv1", "Run authenticated vulnerability scan"),
            ("pv2", "Triage critical/high findings (CVSS 7+)"),
            ("pv3", "Apply patches to the test group and verify"),
            ("pv4", "Roll out patches to production"),
            ("pv5", "Re-scan and confirm findings are closed"),
        ],
    },
    "access": {
        "name": "Access Review & Offboarding",
        "steps": [
            ("ac1", "Disable the user's accounts and SSO sessions"),
            ("ac2", "Revoke VPN, API keys and SSH keys"),
            ("ac3", "Rotate shared secrets the user knew"),
            ("ac4", "Collect laptop and tokens, and wipe devices"),
            ("ac5", "Review privileged-group membership"),
        ],
    },
}
STEP_NAMES = {sid: n for p in PROCESSES.values() for sid, n in p["steps"]}
```

$ cat server.py

- POST /sync: accepts events, returns new ones. UNIQUE event_id makes retries harmless.
- Each event stores a SHA-256 hash of itself plus the previous hash: a chain.
- GET /verify re-checks the chain and reports where it breaks.
- POST /api/processes: new. Checks the name and steps, then saves the checklist to custom_processes.
- GET /api/state: merges built-in and added checklists, with per-user stats. First completion of a step wins.

server.py > add_process

```python
@app.post("/api/processes")
def add_process(p: NewProcess):
    name = " ".join(p.name.split())
    steps = [s for s in (" ".join(x.split()) for x in p.steps) if s]
    if not 1 <= len(name) <= 80:
        raise HTTPException(400, "Give the process a name (up to 80 characters).")
    if not 1 <= len(steps) <= 20:
        raise HTTPException(400, "Add between 1 and 20 steps.")
    if any(len(s) > 120 for s in steps):
        raise HTTPException(400, "Each step must be 120 characters or fewer.")
    with lock:
        if name.lower() in {x["name"].lower() for x in all_processes().values()}:
            raise HTTPException(409, "A process with that name already exists.")
        pid = "c" + uuid.uuid4().hex[:8]
        data = [[f"{pid}_{i + 1}", s] for i, s in enumerate(steps)]
        db.execute("INSERT INTO custom_processes(pid,name,steps,created_at) VALUES(?,?,?,?)",
                   (pid, name, json.dumps(data), time.time()))
        db.commit()
    return {"id": pid}


@app.get("/api/state")
def state():
    with lock:
        rows = db.execute("SELECT seq,step_id,user,device,happened_at,hash FROM events "
                          "ORDER BY seq").fetchall()
        ok, broken = verify_chain()
        seen = dict(devices)
        procs_def = all_processes()
    now = time.time()
    first = {}
    for r in rows:
        first.setdefault(r[1], r)

    step_names = {sid: n for p in procs_def.values() for sid, n in p["steps"]}
    step_proc = {sid: pid for pid, p in procs_def.items() for sid, _ in p["steps"]}
```

$ ls dashboard/

home

Progress ring, stats, chain status and the audit log.

processes

Open a checklist to see who did each step.

add process

Build a new checklist with a live preview.

accounts

Each person’s steps, devices and last activity.

browser -> http://localhost:8000 · refreshes every 2 seconds

$ ./lighthouse --add-process

1. fill the form

On the Add process page, name the checklist and list its steps. A live preview updates as you type.

2. server checks

POST /api/processes trims the text, allows 1 to 20 steps and rejects a duplicate name.

3. all see it

It is saved with a new id, merged into /api/state and tagged Added on the Processes page.

$ app.py # part 1: Store

- complete_step updates the step and queues an outbox row in one transaction: both or neither.
- sync_once pushes the outbox, then pulls events newer than a saved since token.
- Outbox rows are deleted only after the server confirms them.
- sync_loop retries with backoff while offline. Merge rule: completed wins.

app.py > Store > complete_step

```
421     def complete_step(self, step_id):
422         now = time.time()
423         with self.lock, self.db:
424             self.db.execute("UPDATE steps SET done=1, done_by=?, done_at=? WHERE id=? AND done=0",
425                             (self.user, now, step_id))
426             self.db.execute("INSERT INTO outbox VALUES(?,?,?,?)",
427                             (str(uuid4()), step_id, self.user, now))
428         self.wake.set()
429
430     def step_states(self):
431         with self.lock:
432             rows = self.db.execute("SELECT id,done,done_by,done_at FROM steps").fetchall()
433         return {sid: (bool(done), by, at) for sid, done, by, at in rows}
434
435     def pending(self):
436         with self.lock:
437             return self.db.execute("SELECT COUNT(*) FROM outbox").fetchone()[0]
438
439     def _since(self):
440         row = self.db.execute("SELECT value FROM meta WHERE key='since'").fetchone()
441         return int(row[0]) if row else 0
442
443     def sync_once(self):
444         if self.simulate_offline:
445             self.connected = False
446             return False
447         with self.lock:
448             pending = self.db.execute(
449                 "SELECT event_id,step_id,user,happened_at FROM outbox").fetchall()
450             since = self._since()
451         payload = {"device": self.device, "since": since,
452                    "events": [dict(event_id=e, step_id=s, user=u, happened_at=t)
453                               for e, s, u, t in pending]}
454         try:
455             r = requests.post(self.server, json=payload, timeout=5)
456             r.raise_for_status()
```

$ app.py # part 2: UI + security

- Accounts: random salt, PBKDF2-SHA256 at 600,000 iterations, constant-time compare.
- Five wrong passwords lock the account for 30 seconds, and the count survives restarts.
- Animated widgets: CheckBox, StepRow, Toast, Pulse status dot.
- ServerFeed polls /api/state for the team dashboard. An offline toggle demos the sync.
- The desktop client lists the 4 built-in checklists. Added ones show on the web dashboard.

app.py > Accounts

```python
class Accounts:
    ITERS, MIN_LEN = 600_000, 8
    MAX_FAILS, LOCK_SECS = 5, 30

    def __init__(self, path=os.path.join(HERE, "accounts.db")):
        self.db = sqlite3.connect(path, timeout=10)
        self.db.execute("""CREATE TABLE IF NOT EXISTS accounts(
            user TEXT PRIMARY KEY COLLATE NOCASE, salt BLOB, hash BLOB, iters INTEGER,
            fails INTEGER DEFAULT 0, locked_until REAL DEFAULT 0)""")
        self.db.commit()

    @staticmethod
    def _hash(pw, salt, iters):
        return hashlib.pbkdf2_hmac("sha256", pw.encode("utf-8"), salt, iters)

    def any(self):
        return self.db.execute("SELECT 1 FROM accounts LIMIT 1").fetchone() is not None

    def exists(self, user):
        return self.db.execute("SELECT 1 FROM accounts WHERE user=?", (user,)).fetchone() is not None

    def create(self, user, pw):
        salt = os.urandom(16)
        try:
            with self.db:
                self.db.execute("INSERT INTO accounts(user,salt,hash,iters) VALUES(?,?,?,?)",
                                (user, salt, self._hash(pw, salt, self.ITERS), self.ITERS))
        except sqlite3.IntegrityError:
            return False
        return True

    def login(self, user, pw):
        row = self.db.execute("SELECT user,salt,hash,iters,fails,locked_until FROM accounts "
                              "WHERE user=?", (user,)).fetchone()
        if not row:
            self._hash(pw, b"0" * 16, self.ITERS)
```

$ run.py

- Starts server.py and waits until /api/state answers (about 12 seconds max). Dashboard: localhost:8000.
- Opens one app.py window per name: python run.py Ali Bob shows sync live.
- python run.py stop stops a leftover server via server.pid.
- Works from VS too: swaps pythonw.exe for python.exe. Logs go to server.log and client logs.

app.py > Accounts

```python
class Accounts:
    ITERS, MIN_LEN = 600_000, 8
    MAX_FAILS, LOCK_SECS = 5, 30

    def __init__(self, path=os.path.join(HERE, "accounts.db")):
        self.db = sqlite3.connect(path, timeout=10)
        self.db.execute("""CREATE TABLE IF NOT EXISTS accounts(
            user TEXT PRIMARY KEY COLLATE NOCASE, salt BLOB, hash BLOB, iters INTEGER,
            fails INTEGER DEFAULT 0, locked_until REAL DEFAULT 0)""")
        self.db.commit()

    @staticmethod
    def _hash(pw, salt, iters):
        return hashlib.pbkdf2_hmac("sha256", pw.encode("utf-8"), salt, iters)

    def any(self):
        return self.db.execute("SELECT 1 FROM accounts LIMIT 1").fetchone() is not None

    def exists(self, user):
        return self.db.execute("SELECT 1 FROM accounts WHERE user=?", (user,)).fetchone() is not None

    def create(self, user, pw):
        salt = os.urandom(16)
        try:
            with self.db:
                self.db.execute("INSERT INTO accounts(user,salt,hash,iters) VALUES(?,?,?,?)",
                                (user, salt, self._hash(pw, salt, self.ITERS), self.ITERS))
        except sqlite3.IntegrityError:
            return False
        return True

    def login(self, user, pw):
        row = self.db.execute("SELECT user,salt,hash,iters,fails,locked_until FROM accounts "
                              "WHERE user=?", (user,)).fetchone()
        if not row:
            self._hash(pw, b"0" * 16, self.ITERS)
```

$ exit 0

thank you

Questions?